

Reinforcement Learning in Schnapsen

No Author Given

Vrije Universiteit Amsterdam, De Boelelaan 1105, 1081 HV, Amsterdam, The Netherlands

Abstract. The Schnapsen two-player card game presents a challenging environment for intelligent agents due to its strategic complexity and the variability introduced by limited knowledge of the opponent's actions during the initial phase of play. This research investigates how increasing the number of cards in Schnapsen, from the standard 20 to 24, affects the performance of a reinforcement learning bot. By modifying the game engine to include an additional card rank, experiments were conducted in both standard and modified setups. The learning ML bot, which is a custom reinforcement learning agent, was evaluated against five other bots (Rand, Bully, Modified Alphabeta, Modified mini max and Rdeep bots) before and after training in both variants of the game. Their win rates were recorded to assess the adaptability and learning efficiency of the ml bot in varying game conditions. The findings provide insight into the impact of increased game complexity on reinforcement learning strategies and provide a clear overview of the scalability of such approaches in dynamic environments.

Keywords: Schnapsen · Reinforcement Learning · Artificial Intelligence · Q value · Q network · t-test · Wilcoxon signed-rank test.

1 Introduction

The development of artificial intelligence systems has seen significant milestones, especially in the gaming domain. Games have always been an ideal platform for AI research as they offer a combination of structured rules, strategic depth and computational complexity. These characteristics make games a rich terrain for exploring decision-making processes, heuristic evaluations, and learning strategies. Historically, AI achievements in games like Chess and Go have not only pushed the boundaries of computational intelligence, but also challenged human players to refine their skills. IBM's DeepBlue defeating Garry Kasparov in chess in 1997 (Deep Blue) and Google's AlphaGo winning over Lee Sedol in Go in 2016(Borowiec) represent landmark events that demonstrated the potential of AI to master complex systems. Among the many games studied in the realm of AI research, the two player card game Schnapsen stands out for its unique blend of partial and complete information phases. With its structured rules and varying levels of uncertainty, Schnapsen provides an excellent environment for testing and developing AI systems that include reinforcement learning-based

approaches. In the game, there are various aspects that the player should consider. Those are balancing immediate gains with long-term strategies, adapting to changing game states, and guessing their opponent's cards accurately. Those are only a couple of things that make Schnapsen an appealing challenge for AI researchers aiming to evaluate learning-based systems. This research builds on the foundation of using games as a testbed for AI by investigating a variant of Schnapsen that increases the deck size from 20 to 24 cards. The addition of rank 9 in each suit introduces subtle changes to the game's dynamics, increasing its complexity and requiring adaptations in strategy. This study focuses on evaluating how a reinforcement learning bot(Learning ML bot) performs under these altered conditions compared to the standard game. By examining the bot's win rates against various opponents before and after training in both game setups, this research aims to shed light on how increasing the state and action spaces in a game environment affects the adaptability and learning capabilities of reinforcement learning systems.

2 Background Information

2.1 Schnapsen

Schnapsen is a traditional two-player trick-taking card game that is popular in several European countries. The game is played with a 20 card deck which consists of cards ranked from Ace (11 points), Ten (10 points), King (4 points), Queen (3 points), and Jack (2 points) in each of the four suits (Hearts, Diamonds, Clubs, and Spades). The objective is to be the first player to reach 66 points in a round. Each player is dealt five cards, and the remaining ten cards from the "talon," with the top card of the talon turned face-up to designate the trump suit. Players alternate turns, in which the winner of the current trick becomes or stays as the leader. The game consists of two phases. In the first phase, players are not required to play trump cards if they do not have the following suit of the played card. A trick is won by the higher ranked card played in the same suit, playing trump card in which it always beats cards of other suits. Players can also declare a marriage which can be done by making King and Queen in the same suit which will earn them 20 points.(If it is in trump suit, the point will increase to 40) After declaring a marriage, players must lead with the King or Queen during the next trick. Players may exchange one of their trump cards with the trump card in the talon(face-up placed) if they are the leader which is called trump exchange. Once the talon is depleted or closed by a player, the second phase of the game starts and the game shifts to a full-information phase. Players must follow suit and play a trump card if they cannot follow the suit. In the second phase, both players rely on their current hand as they can not draw any more cards. Players accumulate points by winning tricks and earning bonuses from marriages. The value of each trick is calculated as the sum of the card points in that trick. The game ends when a player announces that they have reached or exceeded 66 points, provided that they are correct in their declaration. If not correct, the opponent scores additional points. (Schnapsen)

2.2 24 Schnapsen

The 24 card version of Schnapsen modifies the regular game by introducing a new card rank (9) in each of the four suits. These cards have a point value of 1. This modification increases the total number of cards in the deck from 20 to 24 and slightly changes the dynamics of the game by expanding the range of possible plays and the overall complexity. The 9 of each suit is treated like any other card in terms of gameplay mechanics. It can be played to win or lose a trick based on its rank relative to the opponent's card and the trump suit. Its point value of 1 adds to the scoring potential in each trick, slightly increasing the maximum number of points a player can achieve in a single game. The additional cards also increase the size of the talon and the possible hand combinations, requiring adjustments in strategies for both phases of the game.

3 Research Question

This paper aims to address the question "How does increasing the number of cards in the Schnapsen game affect the performance of a reinforcement learning bot."

4 Experimental set up

4.1 Variables

Independent: The game engine mode. Dependent: The performance of the learning bot. Controlled variable: Ally bot, Opponent bots, number of games between bots, number of training games. Confounding variables: The games by them. Moves that are considering random actions.

4.2 Materials Set Up

-A bot that can learn from past games, in our case it is the learning ml bot. -A game environment where bots can play against each other on a given game engine and number of games -A game training environment that trains the learning bot against its opponent bot. -The opponent bots are rand bot, bully bot, modified Alphabeta bot, modified Minimax bot, and Rdeep bot.

4.3 Opponent bots

Rand bot is a simple bot that is designed to play the Schnapsen game by selecting moves at random. It uses a deterministic random number generator provided during initialization to make its choices. During gameplay, RandBot evaluates the set of valid moves available in any given state. Then selects one move at random from this set using the random number generator. This randomness ensures that the bot's behavior is unpredictable, which makes it useful as a baseline opponent to evaluate other bots that are more complex.

Bully bot is a rule-based bot for Schnapsen that prioritizes aggressive and strategic play using the mechanics of the game. It first checks if it has any cards in the trump suit and plays one of them at random to dominate the trick. If it does not have trump cards and is the follower, it tries to match the suit of the opponent's lead card to remain competitive. If neither condition is met, it selects one of its highest scoring cards at random. It greatly demonstrates the aggressive play style that makes it a unique bot.

Modified Alphabeta bot is a hybrid strategy bot that combines randomness with strategic play, which can change depending on the game's phase. During the first phase, where the talon is still open, it selects moves randomly from the valid options. In the second phase, it switches to the strategic AlphaBetaBot algorithm as the game transitions to a full information state by deciding its moves using the alpha-b beta pruning technique. demonstrates multiple play styles with actual strategic gameplay.

Modified Minimax bot is also a hybrid strategy bot like alphabeta bot that uses random decision making in the first phase and using a minimax algorithm in the second phase.

Rdeep bot is a simulation based bot that evaluates each possible move by simulating multiple future game states up to a specified depth and calculating an average score based on the outcomes of these simulations. For each move, the Rdeep bot creates an assumed state of the game. Then it simulates the game using random moves for the opponent and evaluates the resulting states using heuristics based on the scores of both players. The move with the highest average score across all simulations is selected as the optimal choice. It effectively demonstrates the long term consequences of informed decisions of the bot which makes it a diverse opponent.

4.4 Ally bot

Learning ml bot LearningMLBot is a reinforcement learning robot designed to play the Schnapsen game using Q-learning and a neural network to make strategic decisions. At its core, LearningMLBot encodes the state of the game in a feature vector, which includes information such as scores, trump suit, game phase, talon size, the cards in the bot's hand, won cards, and known opponent cards. This state is passed through a Q-network to compute Q-values for all possible actions. A Q network is a neural network implementation of the Q-learning algorithm which is a reinforcement learning method that is not dependent on predefined models of the environment. Its purpose is to approximate the Q-value function $V(s)$ which represents the expected cumulative reward an agent will receive. Those rewards can be determined by the state (s), taking action (a), and

following an optimal policy thereafter. (Muran) The Bellman equation for deterministic situations which is used to construct the Q-learning as shown in image 1. (Tanwar)

$$V(s) = \max_a (R(s, a) + \gamma V(s'))$$

Image 1: Bellman equation that represents the state value function for deterministic situations.

In our bot, the idea of this formula is used, which is:

$$Q(s, a) = R(s, a) + \gamma \max_{a'} V(s') \quad (1)$$

where,

- $Q(s, a) \rightarrow$ The Q-value of the current state s and action a .
- $r \rightarrow$ The immediate reward received after taking action a .
- $\gamma \rightarrow$ the discount factor, which determines the importance of future rewards which can be between 0 and 1.
- $\max V(s') = Q(s', a') \rightarrow$ The maximum value of the next state s' which is determined by the optional action a' in that state.

The Q-network approximates this function using a neural network. The input to the Q-network is a numerical representation that is a feature vector of the current state of the game (s). The output is a set of Q values in which one for each possible action ' a ' in the action space. The action with the highest Q-value represents the network's prediction of the best move to take. (Baeldung)

During training of our bot, the network minimizes the difference between the predicted Q-values and the target Q-values using a loss function. The loss function in training the bot serves as a measure of how far off the Q-network's predicted Q-values are from the target Q-values. During training, the bot compares the Q-value it predicts for a given state and action to a target Q-value. The target includes the immediate reward from an action and the discounted maximum estimated future reward from the next state. The discount factor determines how much importance is placed on future rewards. The network adjusts its weights to minimize the squared difference between these predicted and target Q-values in a batch of training samples. This process helps the Q-network continuously improve its ability to approximate the optimal Q-values which allows the bot to make better decisions over time. By repeatedly updating the Q-network using sampled experiences, the network learns to approximate the Q-value function for all states and actions.

The learning ML bot uses these Q-values to select moves either by exploiting the best move or exploring randomly based on an epsilon-greedy policy. Epsilon-Greedy Policy The epsilon-greedy policy is a strategy used to balance exploration

(trying new actions to discover their potential rewards) and exploitation (choosing the best-known action based on current knowledge).

beginitemize

With a certain probability (ϵ), it selects a random action from the set of valid actions to explore new possibilities. This helps the agent gather diverse experiences and discover potentially better strategies. With the remaining probability ($1 - \epsilon$), the agent chooses the action with the highest Q-value. Then it uses its current knowledge to make the best decision based on what it has learned so far. (Baeldung) The parameter starts with a high value such as 1.0 to encourage exploration in the early stages. This will motivate the agent to try different actions and gain a broad understanding of the environment. Over time, ϵ gradually decreases (by multiplying it by a decay rate), shifting the agent's focus to exploitation. This balance allows the agent to explore effectively initially while refining its strategy as it learns more accurate Q-values.

As the number of training increases, the bot updates its neural network through experience replay and sampling from a replay buffer to learn from past actions and outcomes. The bot employs a target network to stabilize learning and uses a reward function that leads to favorable outcomes such as scoring points, declaring marriages, and winning tricks. Negative rewards are given for unfavorable actions such as losing a game or playing suboptimal moves that led bot to a losing state in the long term. To ensure efficient training, we used mechanisms like early stopping and hyperparameter tunings like epsilon decay, learning rate and patience to ensure early stopping works when it is necessary. We give a patience score of 5 so that whenever the algorithm understands that the bot is not improving for a significant amount of time, it will drop the training process to prevent overfitting. This architecture enables LearningMLBot to adapt to varying game states and improve its decision making over time.

Mock player perspective Mock Player Perspective class is designed to simulate the state of the game for testing and training purposes without requiring live gameplay. It emulates the behavior of the Player Perspective class by providing mock implementations of its key methods. As seen in 'Code image 2', the class is initialized with a predefined set of cards for the bot's hand, trump card, won cards, opponent's won cards and known cards in the opponent's hand. It also provides details about the current game phase, trump suit, talon size and whether the bot is the leader. This mock perspective allows Learning ML bot to simulate hypothetical game states during training. For example, the bot can evaluate its potential moves in scenarios where it knows the opponent's cards or when specific game conditions, like a depleted talon, are simulated. This is particularly useful for generating consistent and reproducible results when testing bot decision-making capabilities. Additionally, MockPlayerPerspective enables the evaluation of encoded game states, contributing to the feature extraction process required for Q-learning. Together, LearningMLBot and MockPlayerPerspective form a synergistic system where the bot learns optimal strategies through simu-

lated and real gameplay scenarios, ensuring adaptability and robust performance across varied game conditions.

```
class MockPlayerPerspective(PlayerPerspective):
    def __init__(self):
        deck_generator = SchnapsenDeckGenerator()
        all_cards = list(deck_generator.get_initial_deck())

        self._hand = Hand(cards=[all_cards[0], all_cards[1]])
        self._trump_card = all_cards[2]
        self._won_cards = Hand(cards=[all_cards[3]])
        self._opponent_won_cards = Hand(cards=[all_cards[4]])
        self._known_opponent_cards = Hand(cards=[all_cards[5]])

        self._my_score = Score(direct_points=0, pending_points=0)
        self._opponent_score = Score(direct_points=0, pending_points=0)
        self._trump_suit = Suit.HEARTS
        self._phase = GamePhase.ONE
        self._talon_size = 10
        self._is_leader = True
```

Code image 2: An image of code that shows the initialization of an hypothetical game perspective.

4.5 Game Training Environment

The training function iteratively trains LearningMLBot by playing a specified number of games against an opponent using the provided game engine. During training, the bot updates its target network periodically (every 100 games) and trains its Q network using experiences from its replay buffer if enough data are available. Training stops early if an early stop condition is met, reducing the chance of overfitting and ensuring efficient use of resources.

4.6 Procedure

- 1 - Play 10,000 games between the learning bot and randbot before training in a regular Schnapsen game engine and record the win rate result
- 2 - Play 3000 training games between the learning bot and the opponent bot that is played against the learning bot 10000 times before training.
- 3 - Play 10,000 games between the learning bot and randbot after training the Schnapsen game engine and record the winrate results.
- 4 - Compare the win rate results in Step 1 and Step 3 and record it in a different section.
- 5 - Repeat steps 1 to 4 with different opponent bots.
- 6 - Repeat steps 1 to 5 but with the twenty-four Schnapsen game engine
- 7 - Compare the special results recorded in each of the steps for each specific bot.(comparing the special results between the learning bot and randbot with regular Schnapsen and twenty-four Schnapsen)

Table 1: Shows the win rates of learning bot against other bots in different game modes before and after training (shown by pre-train for before training and post-train for after training)

Opponent bots	Rand bot		Bully Bot		Modified Alphabeta bot		Modified Minimax bot		Rdeep bot	
	Pre train	Post train	Pre train	Post train	Pre train	Post train	Pre train	Post train	Pre train	Post train
Regular Schnapsen	63.16 %	63.33 %	52.43 %	52.78 %	54.73 %	54.27 %	53.84 %	53.79 %	18.42 %	18.51 %
Twenty Four Schnapsen	59.51 %	60.28 %	46.95 %	47.12 %	53.14 %	53.34 %	53.03 %	54.05 %	16.36 %	16.15 %

Table 2: Shows the win rate gaps of Learning bot against each opponent in different game modes.(win rate gap = Post train winrate - pre train win rate)

Opponent bots	Rand bot	Bully Bot	Modified Alphabeta bot	Modified Minimax bot	Rdeep bot
Game modes	Win rate gap	Win rate gap	Win rate gap	Win rate gap	Win rate gap
Regular Schnapsen	0.14 %	0.35 %	-0.64 %	-0.05 %	0.09 %
Twenty Four Schnapsen	0.77 %	0.17 %	0.20 %	1.02 %	-0.21 %

5 Results

5.1 Data

'Table 1' shows the data of the winning ml bots of the learning ml against the 5 different opponent bots in 10,000 games in both different game engines before and after training, where the pre means before training and the post means after training, whereas 'Table 2' shows the difference between the winning rates of the different games.

5.2 Statistical Testing

The null hypothesis for the statistical tests is: 'The learning performance of Learning ML Bot is not significantly affected by the number of cards.' We decided to use two different tests to find the p-value of our data. Paired t-test and Wilcoxon signed-rank test. The formula for the paired t test is shown in 'image 2' where:

- D — The mean of the differences between paired observations.
- σ — The standard deviation of the differences.
- n — The number of paired observations.

$$t = \frac{\bar{d} - 0}{\hat{\sigma}/\sqrt{n}}$$

Image 2: The formula of t-value.

$$p = 2 \cdot Pr(T > |t|) \quad (\text{two-tailed})$$

Image 3: The formula of p-value using the formula of t-value.

The p-value for two tailed test (in our test case) is calculated as show in “image 3” Where: T Random variable following tt-distribution with df=n1(the degrees of freedom).

$P(T > |t|) \rightarrow$ Probability of observing a value greater than the absolute value of tt in the tt distribution. (Paired T-test)

The formula of the Wilcoxon signed rank test is shown in steps below:

- $d_i = x_i - y_i$
- Signed ranks = $(\text{sign}(d_i) \times \text{rank}(|d_i|))$
- $R+ = \Sigma(\text{positive signed ranks})$
- $R- = \Sigma(\text{negative signed ranks})$
- $W = \min(R+, R-)$

Where:

- d_i — The difference between two data
- $\text{signs}(d_i)$ - Represents the sign of the difference between paired observations
- $\text{rank}(|d_i|)$ — Represents the rank of the absolute difference, starting from 1 for the smallest.
- $\Sigma(\text{positive signed ranks})$ - The sum of absolute differences’ ranks, (represents the total contribution of negative differences)
- $\Sigma(\text{negative signed ranks}) \rightarrow$ The sum of absolute differences’ ranks, (represents the total contribution of negative differences)
- W Measures the deviation from the symmetry between positive and negative differences. (Smaller values of W indicate stronger evidence against the null hypothesis.) (McClenaghan)

In “Code image 1”, both test formulas are calculated with given data and a single function for each of them. Data are taken from “Table 2”.

```
import numpy as np
from scipy.stats import ttest_rel, wilcoxon

regular_learning_gains = np.array([0.14, 0.35, -0.46, -0.05, 0.09])
schnapsen24_learning_gains = np.array([0.77, 0.17, 0.20, 1.02, -0.21])

stat, p_value = ttest_rel(regular_learning_gains, schnapsen24_learning_gains)
print(f"Paired t-test: stat={stat}, p-value={p_value}")

stat, p_value = wilcoxon(regular_learning_gains, schnapsen24_learning_gains)
print(f"Wilcoxon signed-rank test: stat={stat}, p-value={p_value}")
```

Code image 1: An image of the code that shows the calculation of p-value functions.

```
Paired t-test: stat=-1.4247535759576122, p-value=0.22734877008459672
Wilcoxon signed-rank test: stat=3.0, p-value=0.3125
```

Code output 1: Outputs the p-values of both Wilcoxon signed-rank test and paired t-test values.

According to the results of both tests that are shown in the 'code image 1', the p values are greater than the commonly used significance level 0.05, which means that we fail to reject the null hypothesis. This means that there is no statistically significant evidence to suggest that increasing the number of cards (24-card Schnapsen) significantly affects the learning performance of Learning ML bot compared to regular Schnapsen.(Beers)

6 Conclusion

This study aimed to evaluate the impact of increasing the number of cards in Schnapsen on the learning performance of the Learning ML bot. Statistical tests (paired t tests and Wilcoxon signed rank tests) were conducted to compare the bot's learning gains across the regular and 24-card Schnapsen variants. The results indicated no statistically significant difference in learning performance between the two setups, which suggests that the expanded state and action spaces introduced by the additional cards do not substantially hinder the body's ability to learn and improve through training. These findings highlight the adaptability of reinforcement learning in handling increased game complexity while maintaining its robust performance.

7 Future Work

Future research can explore several avenues to expand on these findings. One promising direction is to increase the number of games and bots tested to improve statistical power and further generalize the results. By only playing 3000 games as training, the bot only managed to gather limited data. Furthermore, incorporating more complex reinforcement learning architectures, such as deep Q-networks or multi-agent reinforcement learning, could provide deeper

insights into how AI adapts to complex environments. In our experiment, LearningMLBot used a single-agent reinforcement learning approach with a basic Q-network, which demonstrated its slight effectiveness to both regular and expanded Schnapsen variants. However, in a complex game like Schnapsen, we would rely better on the more advanced architectures that are capable of handling larger state and action spaces efficiently for our bot.

Finally, examining the bot's decision-making strategies using interpretability techniques could provide valuable insights into how it navigates the trade-offs between exploration and exploitation in dynamic environments. In our experiment, the internal reasoning behind the bot's choices remains unclear even with debugging statements. Implementing interpretability methods, such as visualizing Q-value distributions, could help identify how the bot evaluates its options.

References

1. Author Name. *Q-Learning*. Retrieved from <https://www.geeksforgeeks.org/q-learning-in-python/>, accessed 2025-01-27.
2. Jesse Clifton1, Eric Laber1. *Q Learning: Theory and Applications*. Retrieved from <https://www.annualreviews.org/content/journals/10.1146/annurev-statistics-031219-041220>, accessed 2025-01-27.
3. Jacob Murel. *What is Reinforcement Learning*. Retrieved from <https://www.ibm.com/think/topics/reinforcement-learning>, accessed 2025-01-27.
4. Sanchit Tanwar. *Bellman Equation and Dynamic Programming*. Retrieved from <https://medium.com/analytics-vidhya/bellman-equation-and-dynamic-programming-773ce67fc6a7>, accessed 2025-01-27.
5. Baeldung. *Epsilon Greedy Learning*. Retrieved from <https://www.baeldung.com/cs/epsilon-greedy-q-learning>, accessed 2025-01-27.
6. Brian Beers. *P-Value: What It Is, How to Calculate It, and Examples*. Retrieved from <https://www.investopedia.com/terms/p/p-value.asp>, accessed 2025-01-27.
7. Author Name. *Paired T-test*. Retrieved from <https://www.statisticssolutions.com/free-resources/directory-of-statistical-analyses/paired-sample-t-test/>, accessed 2025-01-27.
8. Elliot McClenaghan. *The Wilcoxon Signed-Rank Test*. Retrieved from <https://www.technologynetworks.com/informatics/articles/the-wilcoxon-signed-rank-test-370384>, accessed 2025-01-27.
9. Author Name. *Deep Blue*. Retrieved from <https://www.ibm.com/history/deep-blue>, accessed 2025-01-27.
10. Stewen Borowiech. *AlphaGo Seals 4-1 Victory over Go Grandmaster Lee Sedol*. Retrieved from <https://www.theguardian.com/technology/2016/mar/15/googles-alphago-seals-4-1-victory-over-grandmaster-lee-sedol>, accessed 2025-01-27.
11. Author Name. *Schnapsen*. Retrieved from <https://www.pagat.com/marriage/schnaps.html>, accessed 2025-01-27.